



MEMORIA CUARTA PRACTICA: PROCESADOR DE LENGUAJE EN HASKELL

Acedo Blanco, Alvaro
Czepiel, Daniel

2 de junio de 2026

Índice

1. ¿Qué es un procesador?	2
1.1. Objetivo de la práctica	2
1.2. Módulos de un procesador	2
1.3. Ámbito de la práctica	2
1.4. Características del lenguaje	3
2. Diseño del analizador léxico	3
2.1. Tokens	3
2.2. Gramática	4
2.3. Diagrama del autómata y acciones semánticas	5
2.4. Descripción de las Acciones Semánticas	5
2.5. Errores	6
3. Diseño del analizador sintáctico	7
3.1. Gramática	7
3.2. Tabla	7
3.3. Errores	9
4. Diseño del gestor de errores	10
5. Anexo: ficheros de prueba	11
5.1. Ficheros correctos	11
5.1.1. Fichero 1	11
5.1.2. Fichero 2	11
5.1.3. Fichero 3	11
5.1.4. Fichero 4	12
5.1.5. Fichero 5	12
5.2. Ficheros con errores	12
5.2.1. Fichero 1	12
5.2.2. Fichero 2	13
5.2.3. Fichero 3	13
5.2.4. Fichero 4	13
5.2.5. Fichero 5	13

1. ¿Qué es un procesador?

1.1. Objetivo de la práctica

Un procesador se encarga de analizar un fichero que debe de estar escrito en un determinado lenguaje formal, como Java, C, ... (aunque no tiene que ser un lenguaje de programación). Recibe como entrada un fichero a procesar y devuelve otra representación del fichero, más adecuada para el proposito que se persigue, o correcto; o si el fichero contiene errores, un mensaje de error claro que permite corregir el fichero

En el mundo de los lenguajes de programación, es un paso previo a la generación del código objeto para hacer un ejecutable del código.

1.2. Módulos de un procesador

Un procesador se puede dividir principalmente en los siguientes módulos interconectados, los cuales suelen actuar en forma de *pipeline* o cascada, donde la salida de una fase sirve como entrada directa para la siguiente:

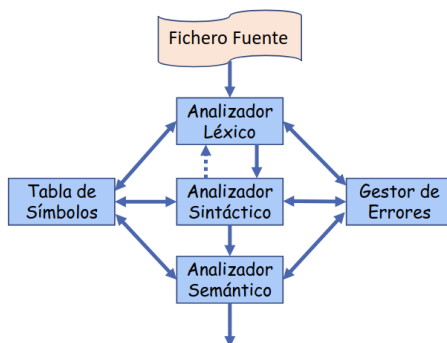


Figura 1: Esquema del procesador.

1. **Analizador léxico (Scanner):** Es la primera fase del proceso. Su objetivo es leer el fichero de entrada carácter a carácter y agruparlos en unidades indivisibles con significado léxico, denominadas *tokens* (como palabras reservadas, identificadores, números, operadores o símbolos de puntuación). Además, se encarga de "limpiar el código", descartando elementos innecesarios para la evaluación lógica, como los espacios en blanco, tabulaciones y comentarios. Internamente, el reconocimiento de estos *tokens* se suele modelar mediante expresiones regulares y Autómatas Finitos Deterministas (AFD).
2. **Analizador sintáctico (Parser):** Recibe el flujo de *tokens* generado por la fase anterior y verifica que su disposición cumple con las reglas gramaticales del lenguaje formal. Su resultado suele ser una representación estructurada, como un Árbol de Sintaxis Abstracta (AST), que refleja la jerarquía de las operaciones.
3. **Analizador semántico:** Toma la estructura generada por el analizador sintáctico y comprueba que, además de estar bien formada, tiene sentido y coherencia. Aquí se realizan tareas como la comprobación de tipos, la verificación del ámbito de las variables o asegurar que una variable ha sido declarada antes de usarse.
4. **Tabla de símbolos:** Es una estructura de datos transversal al resto de analizadores. Actúa como un diccionario donde se registra información vital sobre los identificadores encontrados en el código (variables, funciones, etc.), guardando atributos como su tipo de dato, su ámbito de visibilidad o su posición de memoria.
5. **Gestor de errores:** Es el módulo encargado de interceptar cualquier anomalía durante el proceso de compilación o interpretación. Su misión es recuperarse en la medida de lo posible y emitir mensajes de diagnóstico precisos, indicando la línea y la naturaleza del error para que el programador pueda solucionarlo.

1.3. Ámbito de la práctica

El desarrollo de esta práctica se centra en la implementación de un procesador utilizando el paradigma de programación funcional, concretamente mediante el lenguaje *Haskell* dado en la asignatura. El objetivo principal es construir las fases iniciales descritas anteriormente: análisis léxico y sintáctico; haciendo especial hincapié en el modelado modular.

Se querría haber desarrollado también el analizador semántico sin embargo, debido al tiempo dado para hacer la práctica y su coincidencia con otros trabajos y exámenes nos ha sido imposible terminar la realización de dicho módulo. No obstante, gracias al diseño estrictamente modular empleado en la arquitectura del sistema, se deja la estructura totalmente preparada y abierta para la incorporación futura de esta fase de análisis sin necesidad de reestructurar el núcleo del procesador ya desarrollado.

1.4. Características del lenguaje

Se ha decidido que el procesador realizado se encargará de procesar un lenguaje inventado basado en JavaScript, que tiene características simplificadas para poder hacer la práctica. En la realidad el lenguaje sería más complejo aunque, esto sólo supondría añadir más estados a los autómatas hechos ya que, la lógica subyacente sería la misma.

Generalidades y formato: el lenguaje implementado es *case-sensitive*, esto es que distingue entre mayúsculas y minúsculas; en el que los espacios en blanco, tabulaciones y saltos de línea se tratan como separadores ignorados por el analizador. Las sentencias simples y las declaraciones explícitas finalizan obligatoriamente con un punto y coma (;). El lenguaje admite comentarios de línea mediante el uso del doble operador de división (//).

Estructura de bloques y ámbito: El lenguaje presenta una estructura de bloques delimitados por llaves ({ y }). Maneja los conceptos de:

- **Ámbito global:** Identificadores declarados fuera de cualquier función o variables utilizadas de forma implícita (sin declaración previa), las cuales se asumen por defecto como globales y de tipo entero (`int`).
- **Ámbito local:** Variables declaradas explícitamente mediante la palabra clave `let` en el interior de una función, siendo visibles únicamente desde su declaración hasta el cierre de la misma. No se permite la definición de funciones anidadas.

Tipos de datos y constantes: Se da soporte a los siguientes tipos de datos básicos, sin existencia de conversiones automáticas de tipo implícitas:

- **Entero (`int`):** Representa valores numéricos de 16 bits con signo (rango hasta 32767).
- **Real (`float`):** Valores numéricos de 32 bits con signo. Su representación requiere obligatoriamente al menos un dígito antes y después del punto decimal (ej. 3.0).
- **Cadena (`string`):** Secuencias de caracteres delimitadas por comillas. Ocupan un tamaño fijo en memoria y soportan hasta un máximo de 64 caracteres.
- **Lógico (`boolean`):** Representado mediante las constantes reservadas `true` y `false`.
- **Vacío (`void`):** Utilizado exclusivamente para denotar la ausencia de tipo de retorno o de parámetros en las funciones.

Operadores y precedencia: El sistema modela un conjunto cerrado de operadores agrupados por su funcionalidad (de menor a mayor precedencia):

1. Asignación simple (=).
2. Operador lógico AND (&&).
3. Operadores relacionales de comparación: mayor (>) y mayor o igual (>=).
4. Operadores aritméticos binarios: suma (+) y resta (-).
5. Operadores unarios de alta precedencia: negación lógica (!), autodecremento (-), así como el signo más (+) y menos (-) unarios.

Se permite la utilización de paréntesis de apertura y cierre para alterar el orden natural de evaluación de las expresiones.

Sentencias del lenguaje: Además de las expresiones de asignación, el flujo de ejecución se compone de:

- **Instrucciones de Entrada/Salida:** `read` (para la lectura de datos por teclado hacia una variable) y `write` (para la evaluación y salida por pantalla de expresiones).
- **Estructuras condicionales:** Sentencias de control `if` (sencilla) e `if-else` (compuesta) para la bifurcación del flujo lógico orientada a bloques. Cabe destacar que este dialecto prescinde de bucles iterativos.

Funciones y recursividad: Las funciones se declaran mediante la palabra clave `function` y deben definirse en el ámbito global antes de ser invocadas. El paso de parámetros se realiza estrictamente por valor y se garantiza soporte completo para la recursividad, permitiendo que una función se invoque a sí misma. El retorno de valores se gestiona explícitamente mediante la sentencia `return`.

2. Diseño del analizador léxico

2.1. Tokens

Los tokens que usamos para el analizador léxico son:

El axioma es: S y las reglas de producción son:

$$P = \{ S \rightarrow \text{EOF} \mid \text{del } S \mid dA \mid cD \mid _D \mid "E \mid (\mid) \mid + \mid -I \mid > G \mid \\ \mid ! \mid \&H \mid = \mid ; \mid \{ \mid \} \mid , \mid / J$$

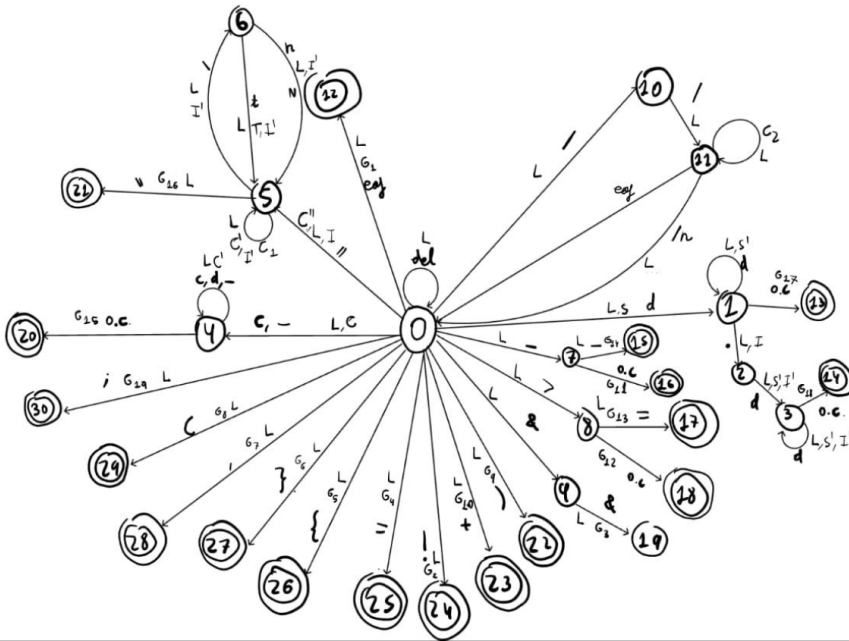
$$\begin{aligned} A &\rightarrow dA \mid .B \mid \lambda \\ B &\rightarrow dC \\ C &\rightarrow dC \mid \lambda \\ D &\rightarrow cD \mid_D dD \mid \lambda \\ E &\rightarrow c_1E \mid \backslash F \mid " \\ F &\rightarrow nE \mid tE \\ G &\rightarrow = \mid \lambda \\ H &\rightarrow \& \\ I &\rightarrow - \mid \lambda \\ J &\rightarrow / K \\ K &\rightarrow c_2K \mid CRS EOF \end{aligned}$$

Siendo:

$$\begin{aligned} \text{del} &= \{\text{espacio, TAB, CR}\} \\ c_1 &= T \setminus \{\backslash, \text{EOF, TAB, "}\} \\ c_2 &= T \setminus \{\text{EOF, TAB}\} \end{aligned}$$

2.3. Diagrama del autómata y acciones semánticas

Hicimos el siguiente autómata de la gramática regular:



Las acciones semánticas son las siguientes:

2.4. Descripción de las Acciones Semánticas

A continuación se detallan todas las acciones semánticas ejecutadas por el autómata durante el proceso de análisis léxico:

G1: Generar Token $\langle \text{eof, -} \rangle$

G2: Generar Token $\langle \text{not, -} \rangle$

G3: Generar Token $\langle \text{and, -} \rangle$

G4: Generar Token $\langle \text{asignación, -} \rangle$

G5: Generar Token $\langle \text{llaveApertura}, - \rangle$
G6: Generar Token $\langle \text{llaveCierre}, - \rangle$
G7: Generar Token $\langle \text{coma}, - \rangle$
G8: Generar Token $\langle \text{parentesisApertura}, - \rangle$
G9: Generar Token $\langle \text{parentesisCierre}, - \rangle$
G10: Generar Token $\langle \text{suma}, - \rangle$
G11: Generar Token $\langle \text{resta}, - \rangle$
G12: Generar Token $\langle \text{mayor}, - \rangle$
G13: Generar Token $\langle \text{mayorIgual}, - \rangle$
G14: Generar Token $\langle \text{autodecremento}, - \rangle$
G15: **if** $p \neq \text{null}$ **then**
 Generar Token $\langle \text{PR_p}, - \rangle$
 else
 $p = \text{BuscarTS}(\text{lexema})$
 if $p = \text{null}$ **then**
 $p = \text{InsertarTS}(\text{lexema})$
 Generar Token $\langle \text{identificador}, p \rangle$
G16: **if** $\text{cont} < 65$ **then**
 Generar Token $\langle \text{cadena}, \text{lexema} \rangle$
 else
 Error(28)
G17: **if** $2^{15} > \text{num}$ **then**
 Generar Token $\langle \text{entero}, \text{num} \rangle$
 else
 Error(21)
G18: $\text{num} = \text{num} \times 10^{-\text{cont}}$
 if $\text{num} < \text{NUM_MAX}$ **then**
 Generar Token $\langle \text{real}, \text{num} \rangle$
 else
 Error(24)
G19: Generar Token $\langle \text{puntoComa}, - \rangle$
L: $\text{car} = \text{leer}()$
C: $\text{lexema} = \text{caracter}$
C': $\text{lexema} = \text{lexema} + \text{caracter}$
C'': $\text{lexema} = ;$ $\text{cont} = 0$
N: $\text{lexema} = \text{lexema} + '\backslash n'$

2.5. Errores

Para los errores hemos tenido en cuenta:

- Caracter desconocido
- El uso incorrecto de comentarios. Del tipo //
- El uso incorrecto del operador and
- La terminación de las cadenas
- Secuencias de escape correctas
- Números reales bien formados
- Esceso de bytes para un tipo de dato

Al encontrarse el analizador léxico con un error este lo que hace es llamar a una función que se encarga de meter en pila el error con su código y mensaje de error.

3. Diseño del analizador sintáctico

3.1. Gramática

Para la gramática hemos utilizado una gramática descendente tabular. Esta tiene la siguiente forma:

El conjunto de de símbolos no terminales es:

NoTerminales = $\{A, B, C, D, E, E1, F, G, H, I, K, L, O, P, Q, R, R1, S, T, U, U1, V, W, X, Z\}$

El axioma es:

Axioma = P

Producciones $P = \{$

$E \rightarrow RE'$	(R1)	$S \rightarrow \text{id } G;$	
$E' \rightarrow \&\&RE'$	(R2)	$S \rightarrow \text{write } E;$	
$E' \rightarrow \lambda$	(R3)	$S \rightarrow \text{read id};$	
$R \rightarrow UR'$		$S \rightarrow \text{return } X;$	
$R' \rightarrow > UR'$	(R5)	$G \rightarrow = E$	(R30)
$R' \rightarrow > = UR'$		$G \rightarrow (L)$	
$R' \rightarrow \lambda$		$L \rightarrow EQ$	
$U \rightarrow VU'$		$L \rightarrow \lambda$	
$U' \rightarrow +VU'$		$Q \rightarrow, EQ$	
$U' \rightarrow -VU'$	(R10)	$Q \rightarrow \lambda$	(R35)
$U' \rightarrow \lambda$		$X \rightarrow E$	
$V \rightarrow +W$		$X \rightarrow \lambda$	
$V \rightarrow -W$		$B \rightarrow \text{if } (E)Z$	
$V \rightarrow !W$		$Z \rightarrow S$	
$V \rightarrow - - W$	(R15)	$Z \rightarrow \{C\}I$	(R40)
$V \rightarrow W$		$I \rightarrow \text{else } D$	
$W \rightarrow \text{id } O$		$I \rightarrow \lambda$	
$W \rightarrow (E)$		$D \rightarrow \text{if } (E)\{C\}I$	
$W \rightarrow \text{entero}$		$D \rightarrow \{C\}$	
$W \rightarrow \text{real}$	(R20)	$B \rightarrow \text{let } T \text{ id};$	(R45)
$W \rightarrow \text{cadena}$		$B \rightarrow S$	
$W \rightarrow \text{true}$		$T \rightarrow \text{int}$	
$W \rightarrow \text{false}$		$T \rightarrow \text{float}$	
$O \rightarrow (L)$		$T \rightarrow \text{boolean}$	
$O \rightarrow \lambda$	(R25)	$T \rightarrow \text{string}$	(R50)
$F \rightarrow \text{function } H \text{ id } (A)\{C\}$	(R51)		
$H \rightarrow T$			
$H \rightarrow \text{void}$			
$A \rightarrow T \text{ id } K$			
$A \rightarrow \text{void}$	(R55)		
$K \rightarrow, T \text{ id } K$			
$K \rightarrow \lambda$			
$C \rightarrow BC$			
$C \rightarrow \lambda$			
$P \rightarrow BP$	(R60)		
$P \rightarrow FP$			
$P \rightarrow \lambda$	(R62)		

3.2. Tabla

Se adjunta en la siguiente página la tabla del analizador sintáctico por motivos de su tamaño, está en su propia página y dada la vuelta para poder caber mejor.

Se tuvo que calcular el follow y next de las reglas para poder hacerla, y la tabla demuestra que la gramática es LL(1), ya que, de no serlo deberían de coincidir en la misma casilla dos o más reglas.

3.3. Errores

Para los errores hemos tenido en cuenta:

- Errores al leer terminales no esperados
- Empezar mal una sentencia
- No poner los atributos de una función
- Declarar de forma incorrecta una función
- No terminar la asignación de una variable
- Expresiones con operandos incorrectas
- El cierre correcto del archivo con un final de bloque o de sentencia

4. Diseño del gestor de errores

Para el gestor de errores hemos decidido hacer recuperación de errores, así el gestor recopila los errores del archivo entero y luego los vuelca una vez terminado de analizar el fichero entero.

Para el contéo del número de línea, creamos una función la cual el analizador léxico llama cada vez que lee el delimitador salto de línea.

5. Anexo: ficheros de prueba

El lenguaje que elegimos para hacer la práctica es C, el cual te obliga a hacer gestión de la memoria dinámica. Hemos tenido en la implementación final varios errores de memoria.

5.1. Ficheros correctos

5.1.1. Fichero 1

```
1  let int a ;
2  let float b ;
3  let boolean bbb ;
4  a = a - c;
5  write a ;
6  write b;
7  write ccc;
```

fichero1.txt

Da un error de memoria el procesador al intentar ejecutar el fichero

TablaTokens:

TablaSimbolos:

Parse:

5.1.2. Fichero 2

```
1  let int x; function float hola (int hola){}
2  hola(x);
3  function void hola2(float f, boolean b){
4      let int i;
5      hola(--i);
6      hola(-i);
7      return ;
8  }
9
10 let boolean y;
11 hola2(hola(x),y);
```

fichero2.txt

Da un error de memoria el procesador al intentar ejecutar el fichero

TablaTokens:

TablaSimbolos:

Parse:

5.1.3. Fichero 3

```
1  function string hello(void){
2      let string hola = "hola";
3      write hola;
4      if(true){
5          return hola;
6      }
7      else{
8          return hello();
9      }
10 }
```

fichero3.txt

Da un error de memoria el procesador al intentar ejecutar el fichero

TablaTokens:

TablaSimbolos:

Parse:

5.1.4. Fichero 4

```
1  let boolean a;  
2  let boolean b;  
3  a = 3 &&( 4 > (2 + 1) )&& !true;  
4  b=a;  
5  let float rrr;  
6  let float ccc;if (rrr >= ccc) b = false;
```

fichero4.txt

TablaTokens:

TablaSimbolos:

Arbol de derivación: PP

```
      P  
      B  
      let  
      T  
      boolean  
      id  
      ;  
P  
  B  
  let  
  T  
  boolean  
  id  
  ;  
P  
  B  
  P
```

5.1.5. Fichero 5

```
1  //f y g no declarados antes, luego son tipo entero  
2  write (f - g);  
3  if(false){  
4    hello();  
5  }  
6  else if(true){  
7    nada = 4;  
8  }  
9  else{}  
10 if (true) read nada;
```

fichero5.txt

TablaTokens:

TablaSimbolos:

Parse:

5.2. Ficheros con errores

5.2.1. Fichero 1

```
1  //Creo que falta poner el void  
2  function float a fun (){  
3  let string tab = 9;  
4  return c;  
5  }  
6  (tab);
```

fichero1.txt

Errores:

5.2.2. Fichero 2

```
1 function int h (int a, float b, string c){
2     if(true){
3         --c;
4         return c;
5     }
6     else{
7         return h(c, b, a);
8     }
9 }
```

fichero2.txt

Errores:

5.2.3. Fichero 3

```
1 function string hola(float a,b){}
2 pues__nada = hola(a,b);
3 let float a;
4 let int b;
5 if(a < b){}
6 else if(!(a-b)){}
```

fichero3.txt

Errores:

5.2.4. Fichero 4

```
1 if(a){
2     return a;
3 }
4 if(3>2>1) then pues__nada = true;
5 let int b;
6 let float c;
7 a=b+c;
```

fichero4.txt

Errores:

5.2.5. Fichero 5

```
1 function int entero (int a, float b){
2     let string c;
3     return c;
4     return a-b;
5     return ;
6 }
```

fichero5.txt

Errores: